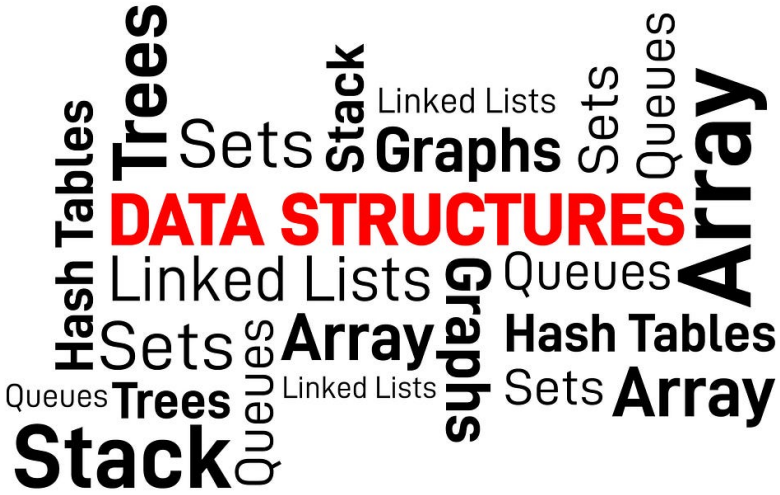


Data Structures

IT623

Dr. Manish Khare

Lecture – 13

Hight Balanced Tree (AVL – Tree)

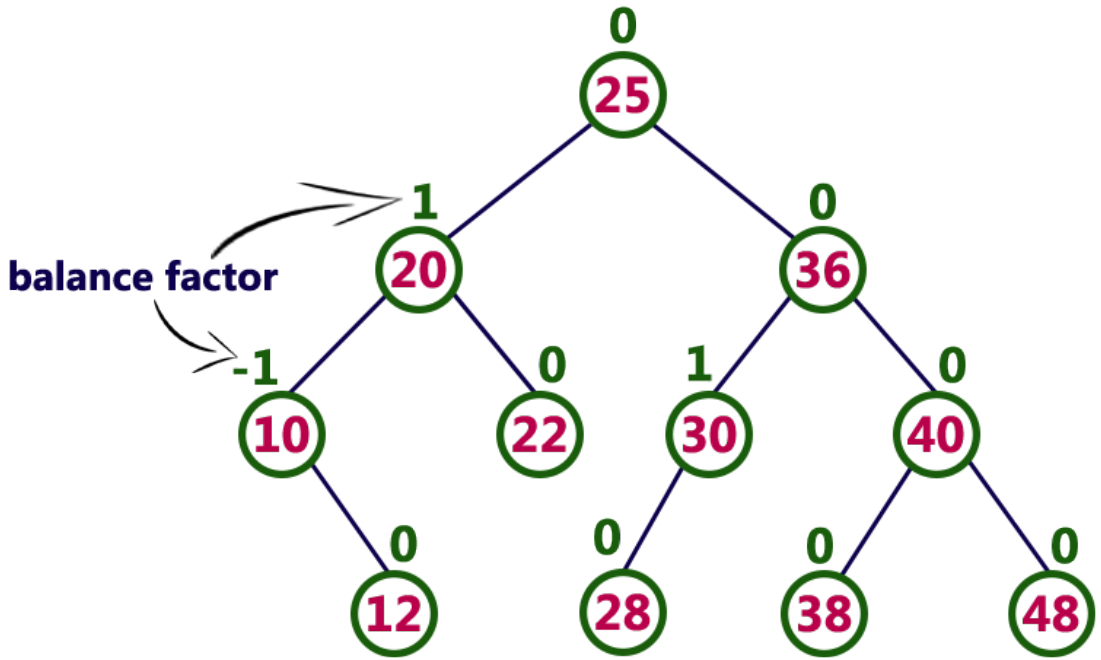
AVL Tree

- AVL tree is a self balanced binary search tree. That means, an AVL tree is also a binary search tree but it is a balanced tree.
- A binary tree is said to be balanced, if the difference between the heights of left and right subtrees of every node in the tree is either -1, 0 or +1.
- In other words, a binary tree is said to be balanced if for every node, height of its children differ by at most one.
- In an AVL tree, every node maintains a extra information known as **balance factor**.
- The AVL tree was introduced in the year of 1962 by G.M. Adelson-Velsky and E.M. Landis.

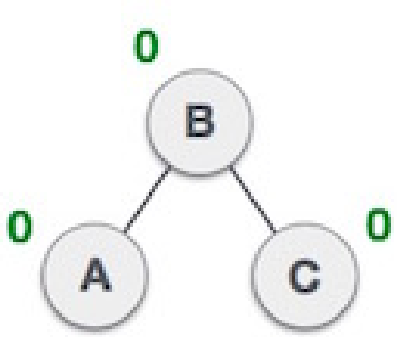
An AVL tree is a balanced binary search tree. In an AVL tree, balance factor of every node is either -1, 0 or +1.

- Balance factor of a node is the difference between the heights of left and right subtrees of that node. The balance factor of a node is calculated either **height of left subtree - height of right subtree** (OR) **height of right subtree - height of left subtree**. In the following explanation, we are calculating as follows...

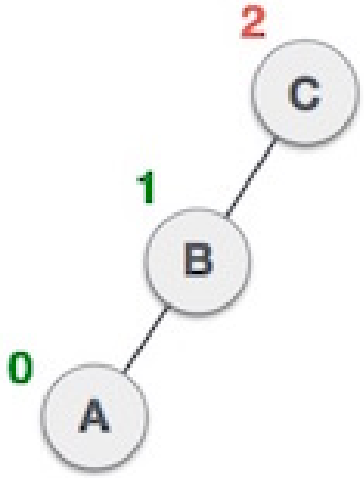
Balance factor = height_of_Left_Subtree – height_of_Right_Subtree



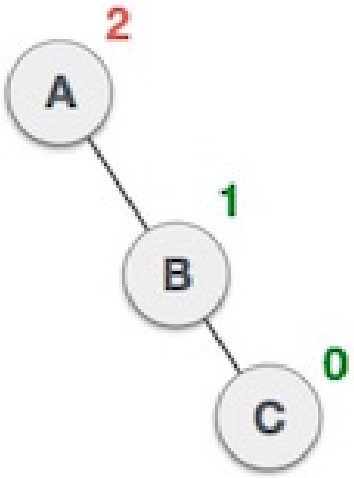
- The above tree is a binary search tree and every node is satisfying balance factor condition. So this tree is said to be an AVL tree.
- **Every AVL Tree is a binary search tree but all the Binary Search Trees need not to be AVL trees.**



Balanced



Not balanced



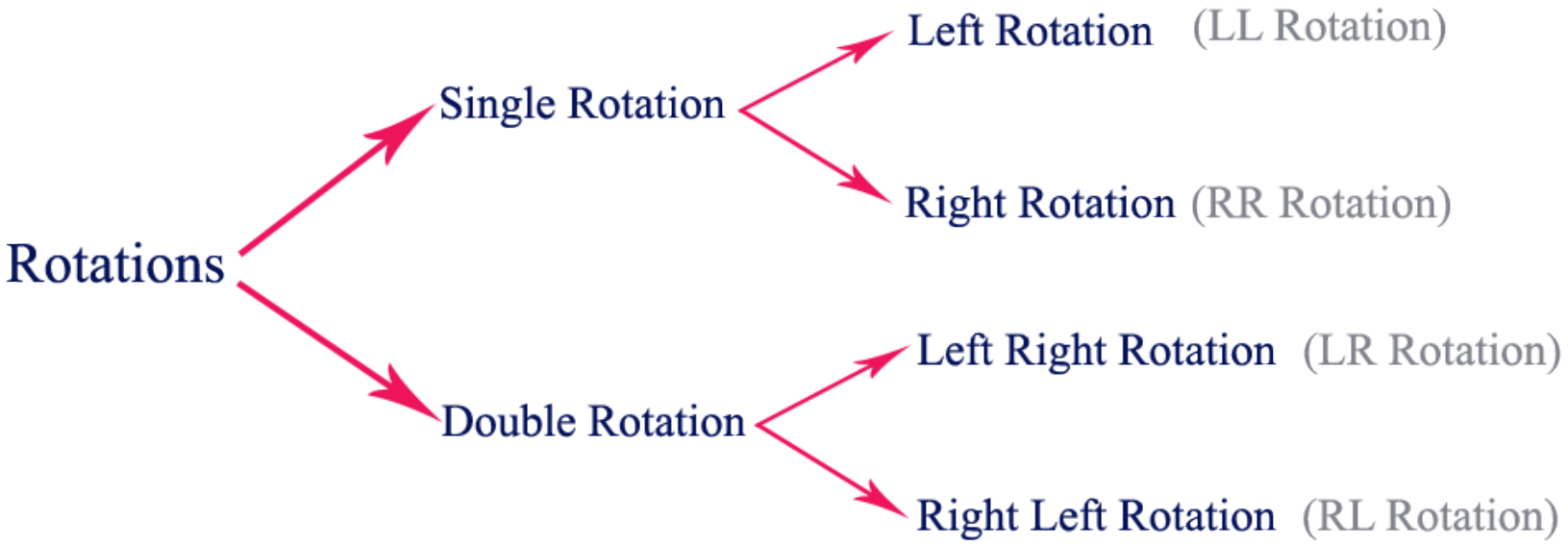
Not balanced

AVL Tree Rotations

- In AVL tree, after performing every operation like insertion and deletion we need to check the **balance factor** of every node in the tree.
- If every node satisfies the balance factor condition then we conclude the operation otherwise we must make it balanced. We use **rotation** operations to make the tree balanced whenever the tree is becoming imbalanced due to any operation.
- Rotation operations are used to make a tree balanced.

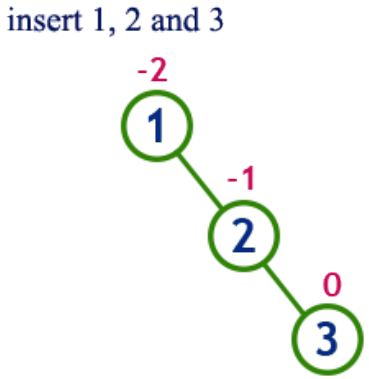
- **Rotation is the process of moving the nodes to either left or right to make tree balanced.**

- There are **four** rotations and they are classified into **two** types.

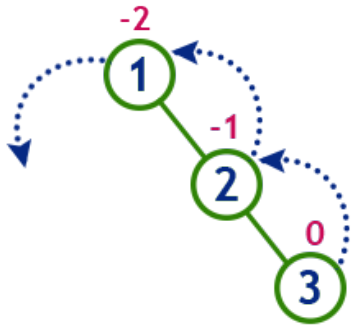


Single Left Rotation (LL Rotation)

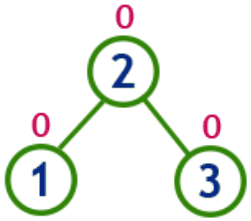
- In LL Rotation every node moves one position to left from the current position.
- To understand LL Rotation, let us consider following insertion operations into an AVL Tree...



Tree is imbalanced



To make balanced we use LL Rotation which moves nodes one position to left

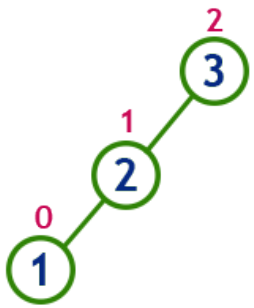


After LL Rotation Tree is Balanced

Single Right Rotation (RR Rotation)

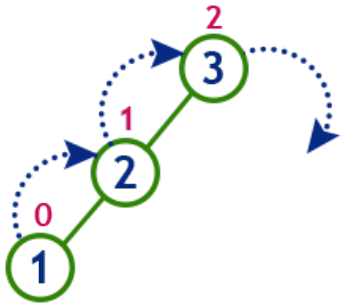
- In RR Rotation every node moves one position to right from the current position.
- To understand RR Rotation, let us consider following insertion operations into an AVL Tree...

ert 3, 2 and 1

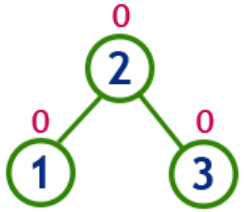


Tree is imbalanced

cause node 3 has balance factor 2



To make balanced we use RR Rotation which moves nodes one position to right

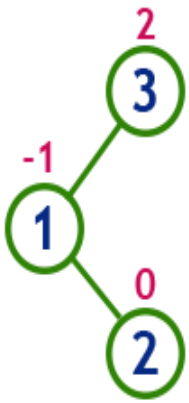


After RR Rotation Tree is Balanced

Left Right Rotation (LR Rotation)

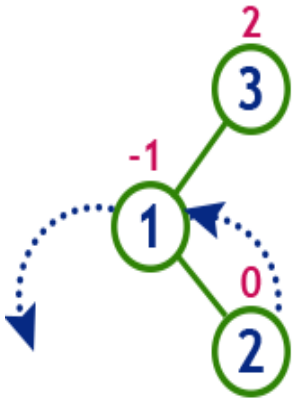
- Double rotations are slightly complex version of already explained versions of rotations.
- The LR Rotation is combination of single left rotation followed by single right rotation.
- In LR Rotation, first every node moves one position to left then one position to right from the current position.
- To understand LR Rotation, let us consider following insertion operations into an AVL Tree...

insert 3, 1 and 2



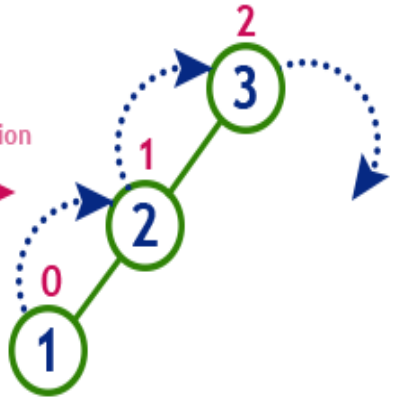
Tree is imbalanced

because node 3 has balance factor 2



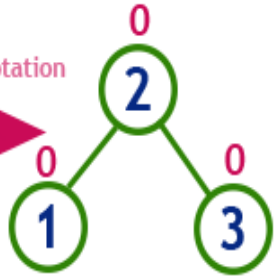
LL Rotation

After LL Rotation



RR Rotation

After RR Rotation

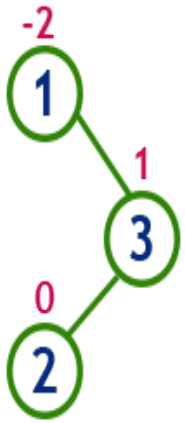


After LR Rotation
Tree is Balanced

Right Left Rotation (RL Rotation)

- The RL Rotation is combination of single right rotation followed by single left rotation.
- In RL Rotation, first every node moves one position to right then one position to left from the current position.
- To understand RL Rotation, let us consider following insertion operations into an AVL Tree...

insert 1, 3 and 2

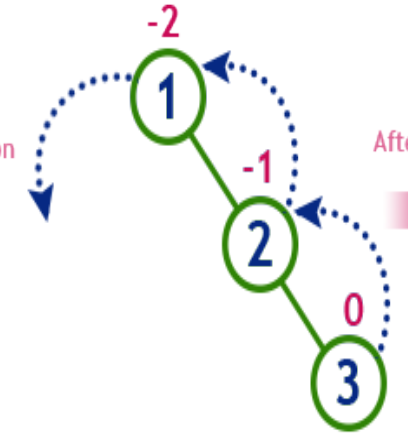


Tree is imbalanced

because node 1 has balance factor -2

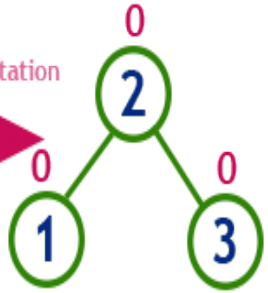
RR Rotation

After RR Rotation



LL Rotation

After LL Rotation



After RL Rotation
Tree is Balanced

AVL Tree Demonstration

Operations on an AVL Tree

- The following operations are performed on an AVL tree...
  - **Search**
  - **Insertion**
  - **Deletion**
  - **Traversal**

Search Operation in AVL Tree

- In an AVL tree, the search operation is performed with **O(log n)** time complexity. The search operation is performed similar to Binary search tree search operation. We use the following steps to search an element in AVL tree...
  - **Step 1:** Read the search element from the user
  - **Step 2:** Compare, the search element with the value of root node in the tree.
  - **Step 3:** If both are matching, then display "Given node found!!!" and terminate the function
  - **Step 4:** If both are not matching, then check whether search element is smaller or larger than that node value.
  - **Step 5:** If search element is smaller, then continue the search process in left subtree.
  - **Step 6:** If search element is larger, then continue the search process in right subtree.
  - **Step 7:** Repeat the same until we found exact element or we completed with a leaf node
  - **Step 8:** If we reach to the node with search value, then display "Element is found" and terminate the function.
  - **Step 9:** If we reach to a leaf node and it is also not matching, then display "Element not found" and terminate the function.

Insertion Operation in AVL Tree

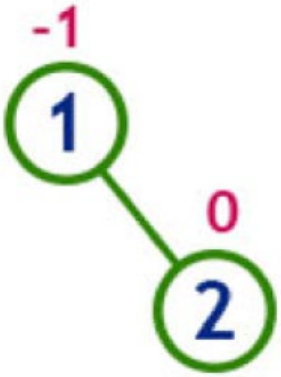
- In an AVL tree, the insertion operation is performed with **O(log n)** time complexity. In AVL Tree, new node is always inserted as a leaf node. The insertion operation is performed as follows...
  - **Step 1:** Insert the new element into the tree using Binary Search Tree insertion logic.
  - **Step 2:** After insertion, check the **Balance Factor** of every node.
  - **Step 3:** If the **Balance Factor** of every node is **0 or 1 or -1** then go for next operation.
  - **Step 4:** If the **Balance Factor** of any node is other than **0 or 1 or -1** then tree is said to be imbalanced. Then perform the suitable **Rotation** to make it balanced. And go for next operation.

- **Construct an AVL Tree by inserting numbers from 1 to 8**

insert 1

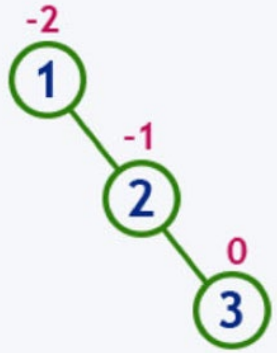
Tree is balanced

insert 2

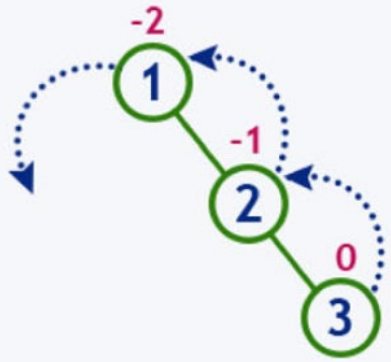


Tree is balanced

insert 3

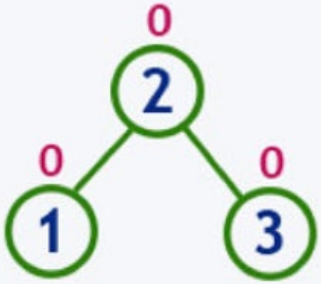


Tree is imbalanced



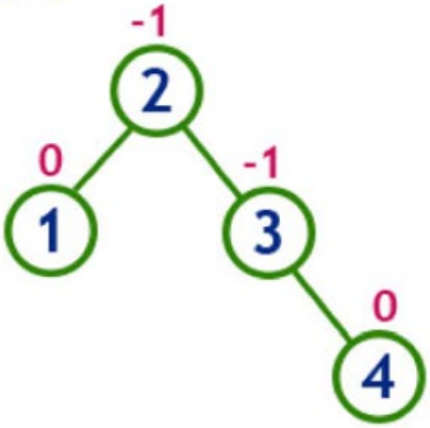
LL Rotation

After LL Rotation

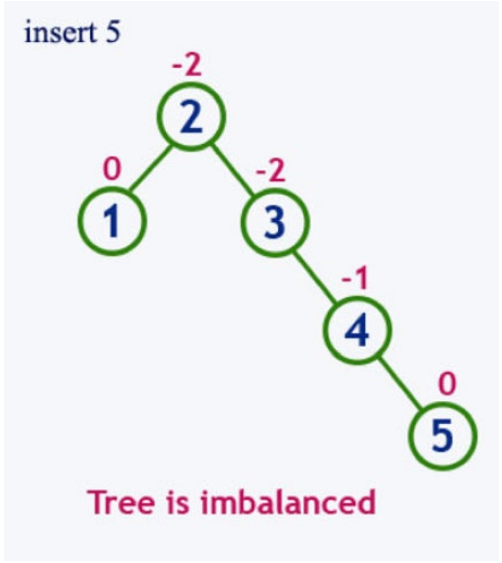


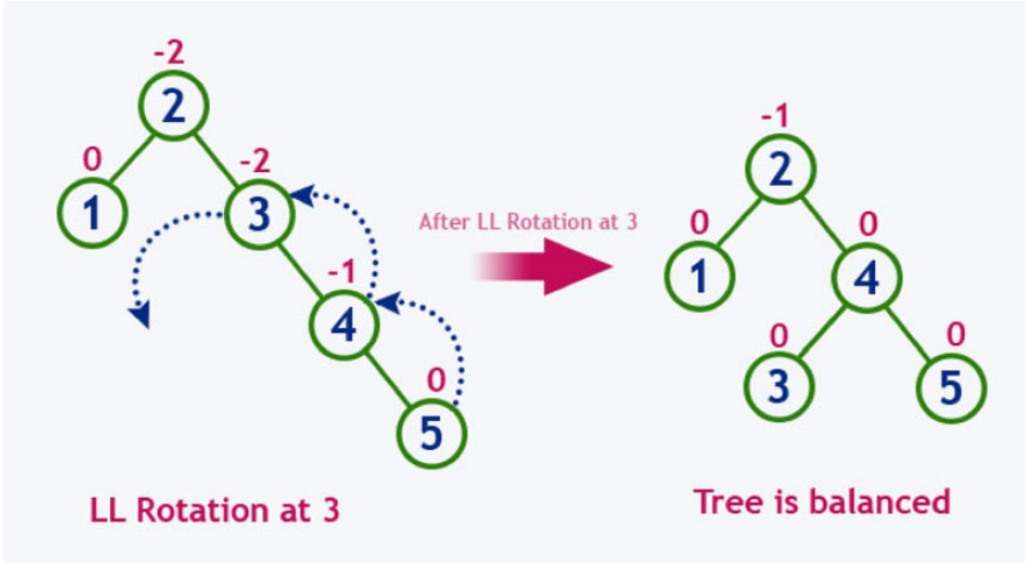
Tree is balanced

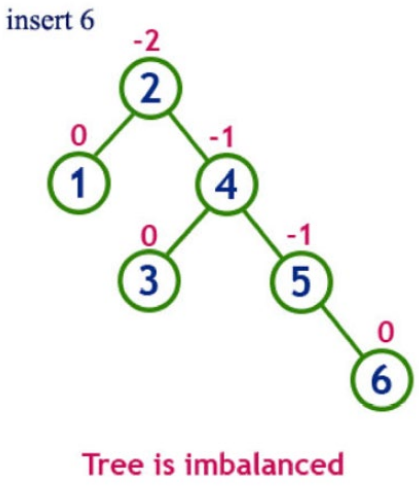
insert 4

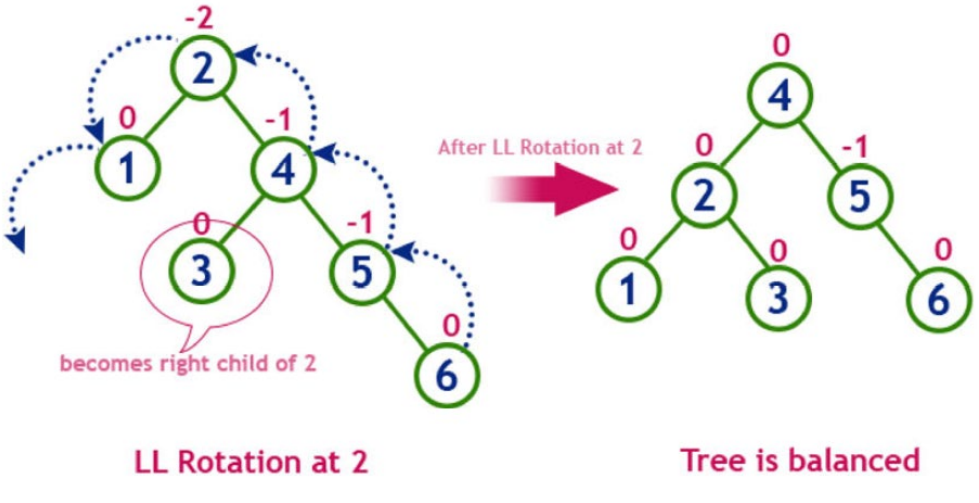


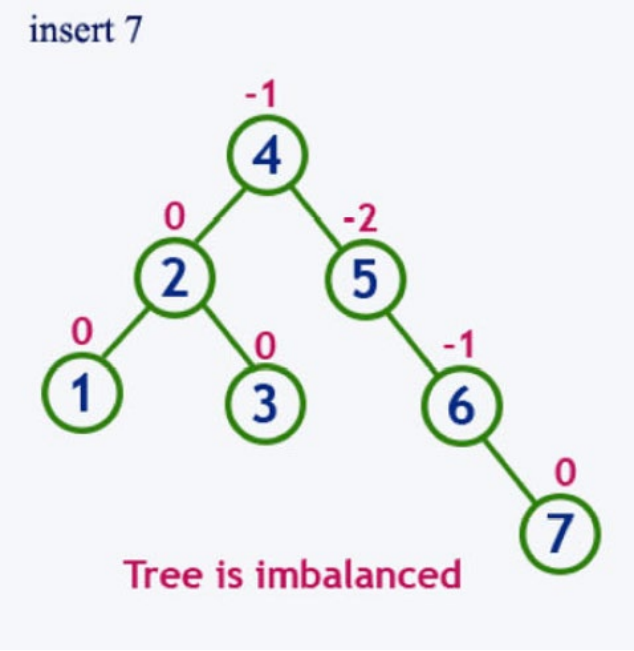
Tree is balanced

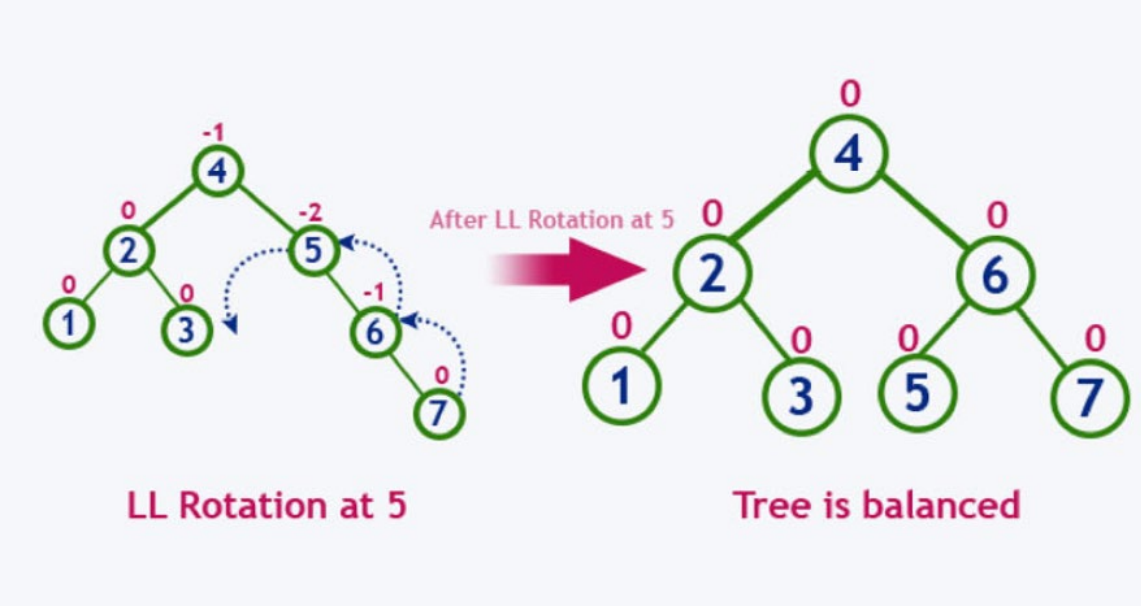

insert 5
-2
2
0
1
-2
3
-1
4
0
5
Tree is imbalanced


-2
2
0
1
-2
3
-1
4
0
5
LL Rotation at 3
After LL Rotation at 3
-1
2
0
1
0
4
0
3
0
5
Tree is balanced

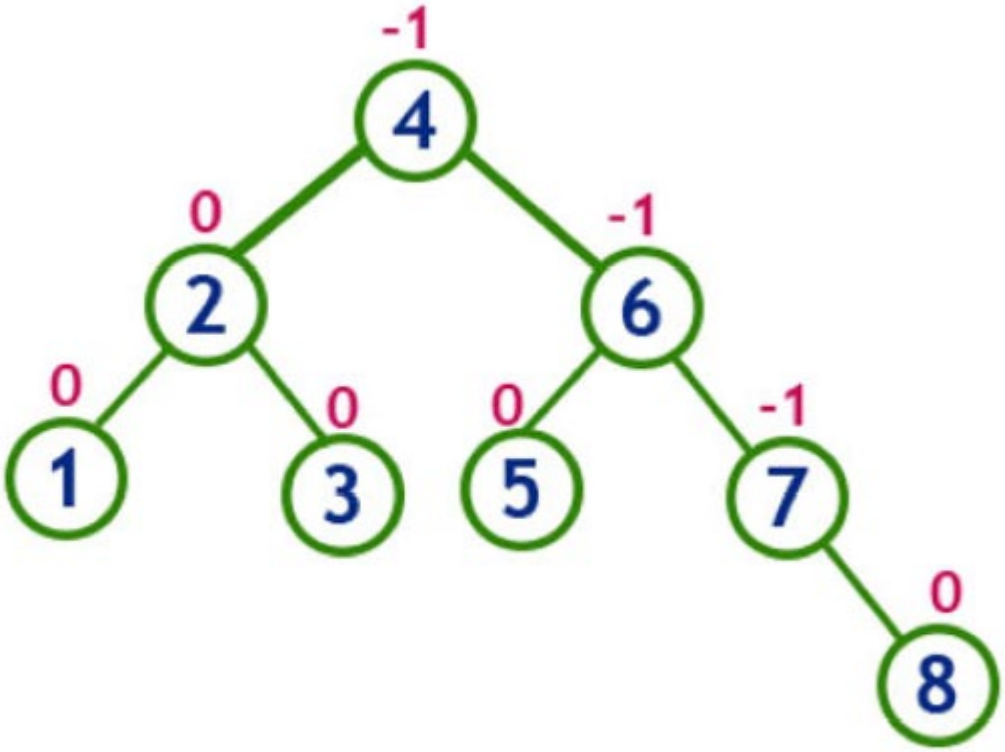

insert 6
-2
2
0
1
-1
4
0
3
-1
5
0
6
Tree is imbalanced


-2
2
0
1
-1
4
0
3
-1
5
0
6
becomes right child of 2
LL Rotation at 2
After LL Rotation at 2
0
4
0
2
-1
5
0
1
0
3
0
6
Tree is balanced


insert 7
-1
4
0
2
-2
5
0
1
0
3
-1
6
0
7
Tree is imbalanced


-1
4
0
2
-2
5
0
1
0
3
-1
6
0
7
LL Rotation at 5
After LL Rotation at 5
0
4
0
2
0
6
0
1
0
3
0
5
0
7
Tree is balanced

insert 8



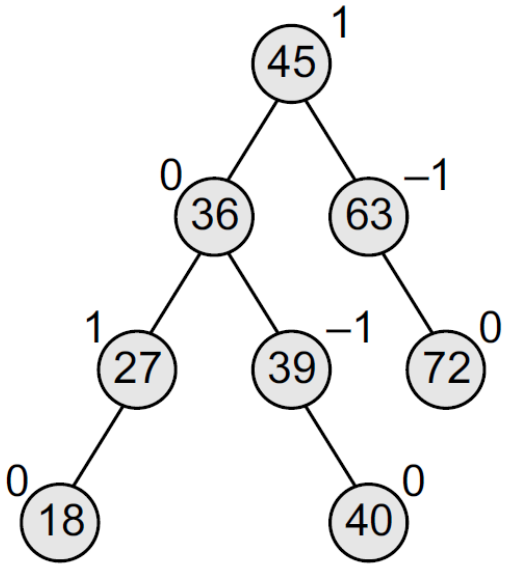
Tree is balanced

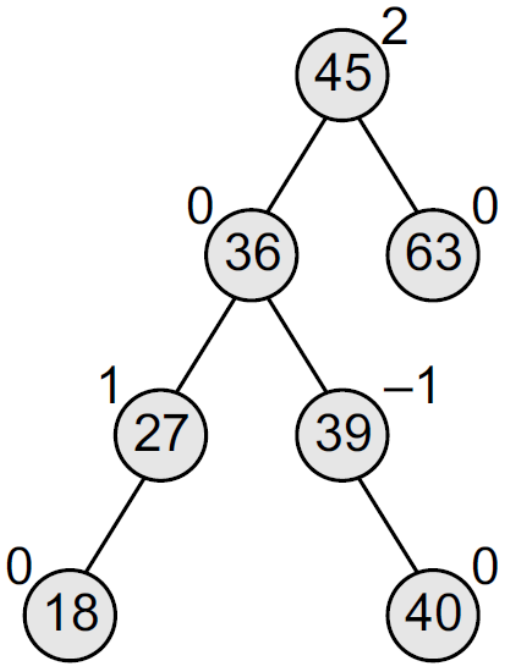
Deletion Operation in AVL Tree

- In an AVL Tree, the deletion operation is similar to deletion operation in BST. But after every deletion operation we need to check with the Balance Factor condition.

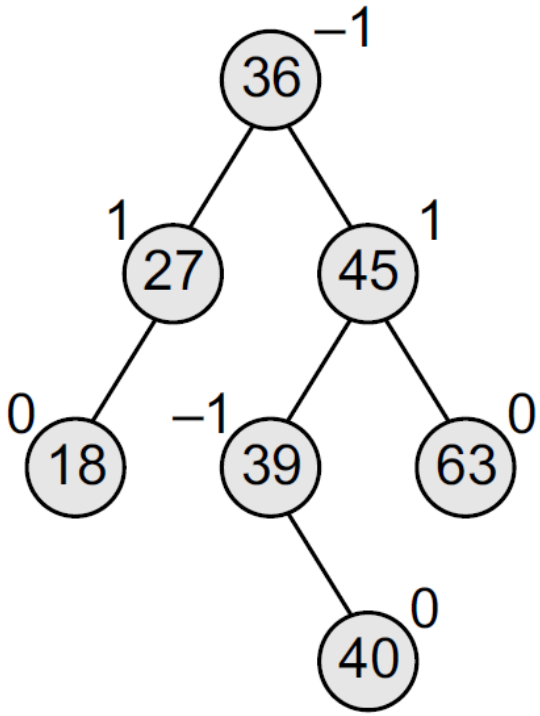
- If the tree is balanced after deletion then go for next operation otherwise perform the suitable rotation to make the tree Balanced.

- Consider the AVL tree given in below Fig. and delete 72 from it.





(Step 1)



(Step 2)

Heap Tree

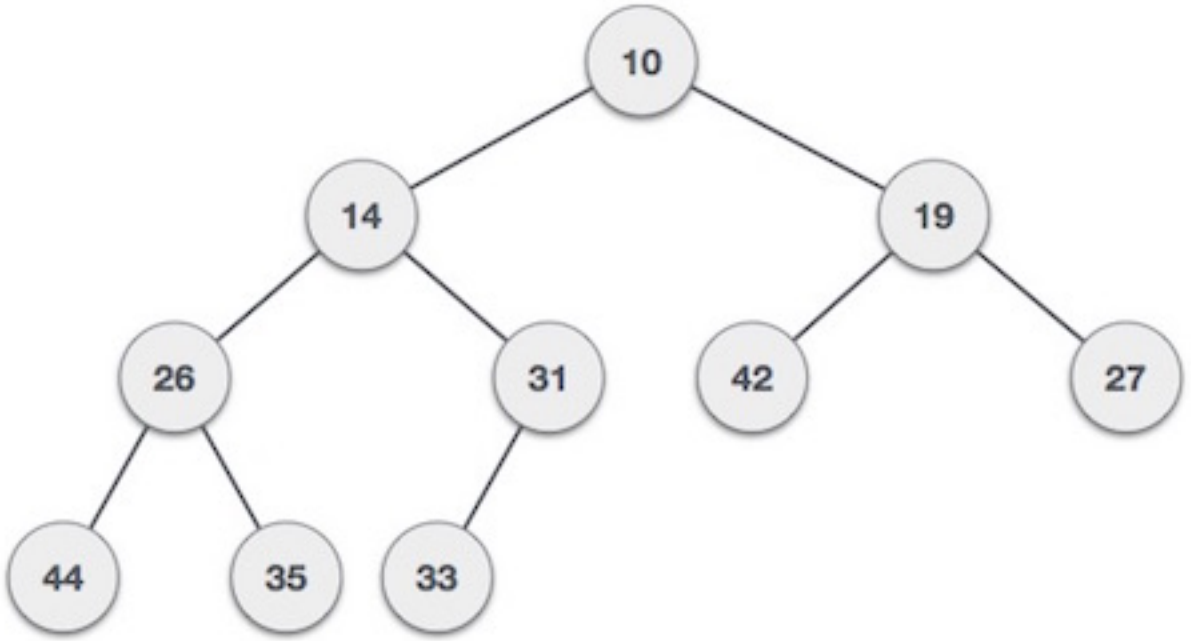
Heap Tree

- Heap is a special case of balanced binary tree data structure where the root-node key is compared with its children and arranged accordingly. If **α** has child node **β** then −

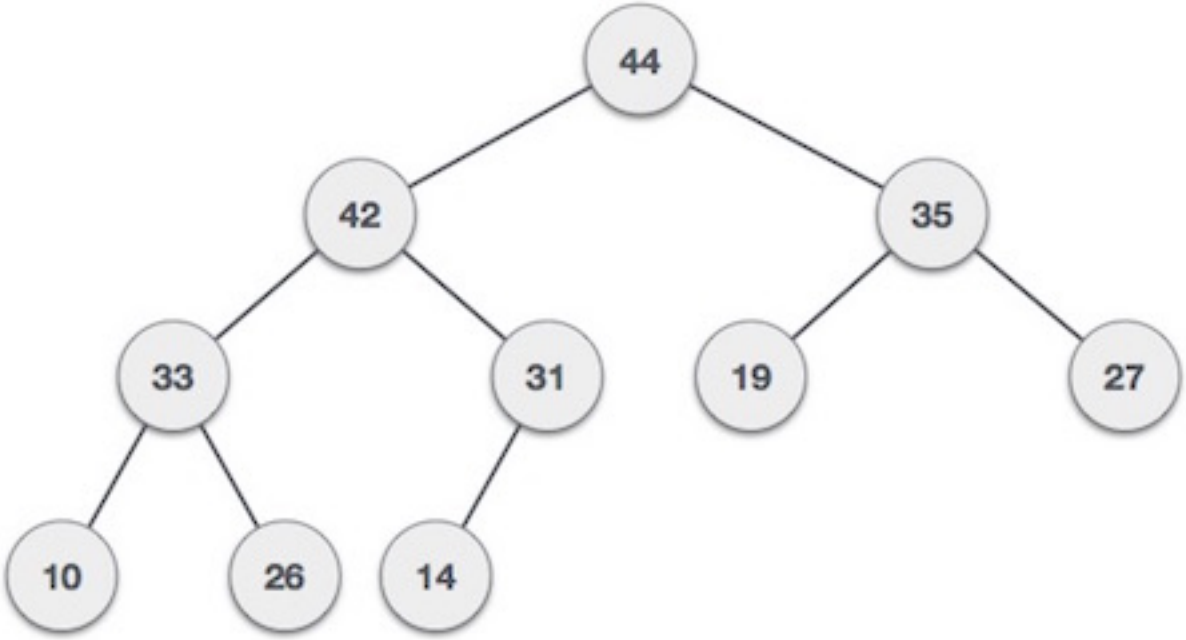
$$\mathbf{key(\alpha) \geq key(\beta)}$$

- Based on this criteria, a heap can be of two types −
  - Min Heap
  - Max Heap

- **Min-Heap** − Where the value of the root node is less than or equal to either of its children.



- **Max-Heap** − Where the value of the root node is greater than or equal to either of its children.



- Every heap data structure has the following properties...

  - **Property #1 (Ordering):** Nodes must be arranged in a order according to values based on Max heap or Min heap.

  - **Property #2 (Structural):** All levels in a heap must full, except last level and nodes must be filled from left to right strictly.

Operations on Heap

- The following operations are performed on a Max/Min heap data structure...
  - **Finding Maximum/Minimum**
  - **Insertion**
  - **Deletion**

Finding Max/Min Value Operation in Max/Min Heap

- Finding the node which has maximum/Minimum value in a max/min heap is very simple.

- In max/min heap, the root node has the maximum/minimum value than all other nodes in the max/min heap.

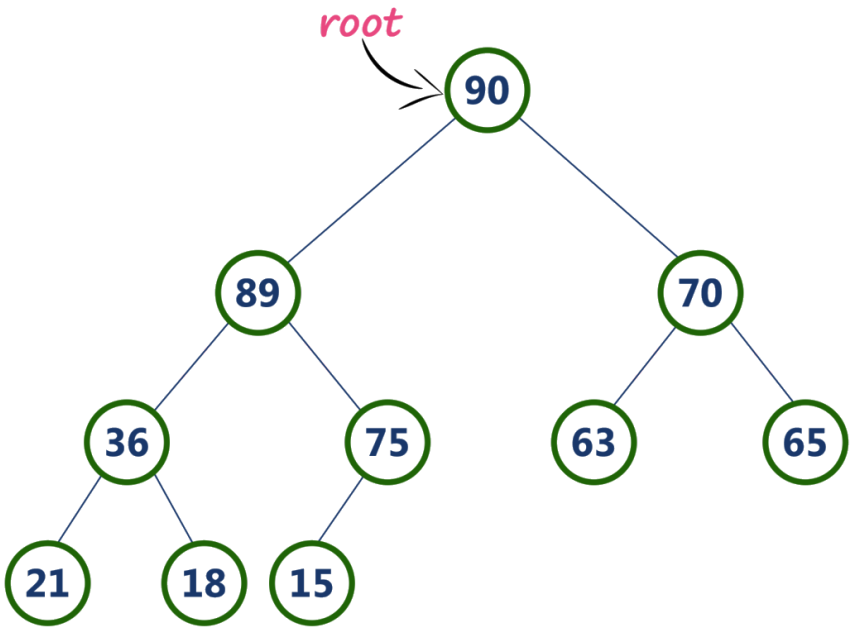
- So, directly we can display root node value as maximum/minimum value in max/min heap.

Insertion Operation in Max Heap

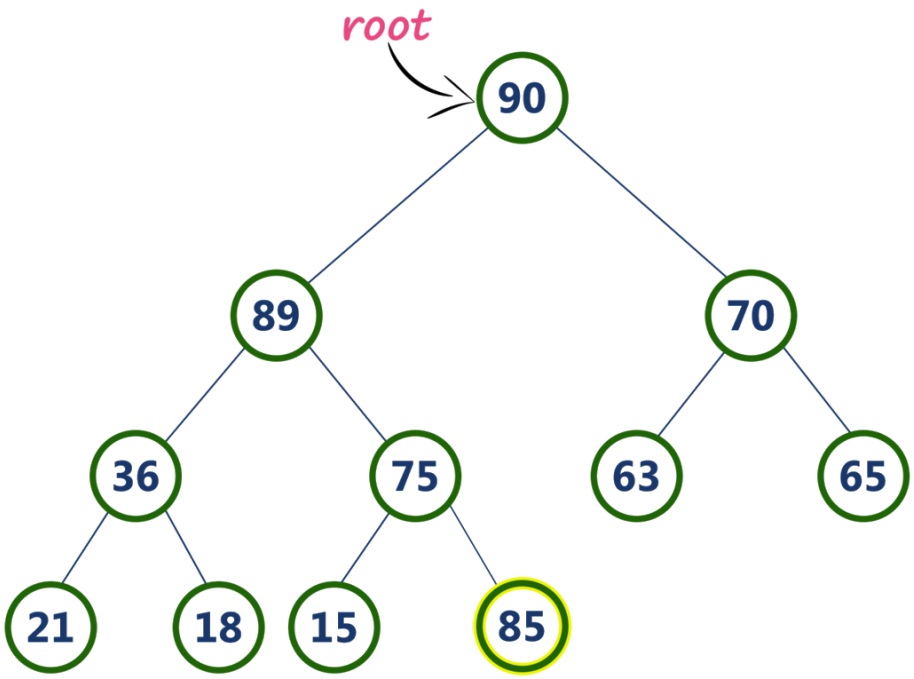
- Insertion Operation in max heap is performed as follows...
  - **Step 1:** Insert the **newNode** as **last leaf** from left to right.
  - **Step 2:** Compare **newNode value** with its **Parent node**.
  - **Step 3:** If **newNode value is greater** than its parent, then **swap** both of them.
  - **Step 4:** Repeat step 2 and step 3 until newNode value is less than its parent nede (or) newNode reached to root.

Insertion Operation in Max Heap - Example

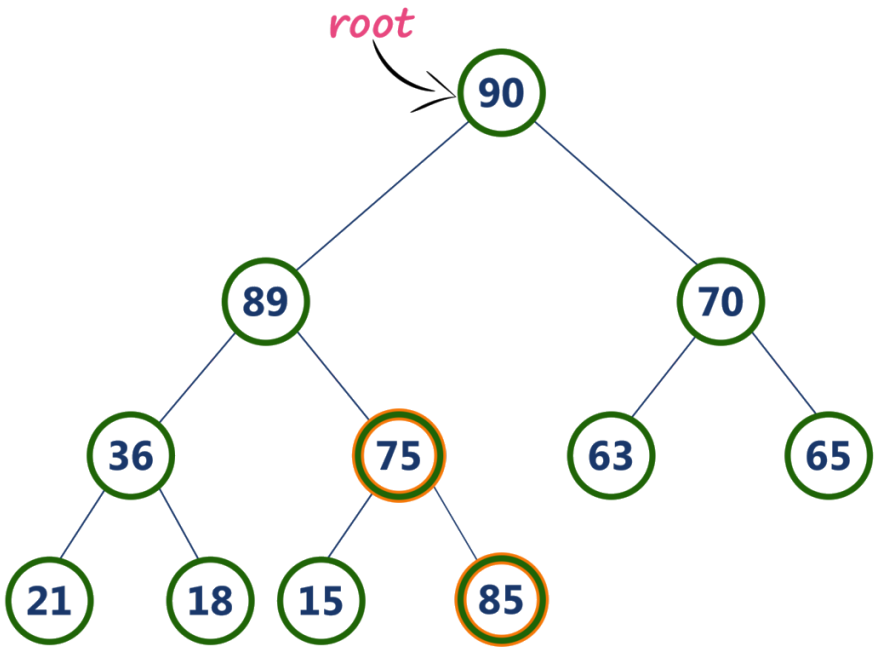
- Consider the following max heap. **Insert a new node with value 85.**



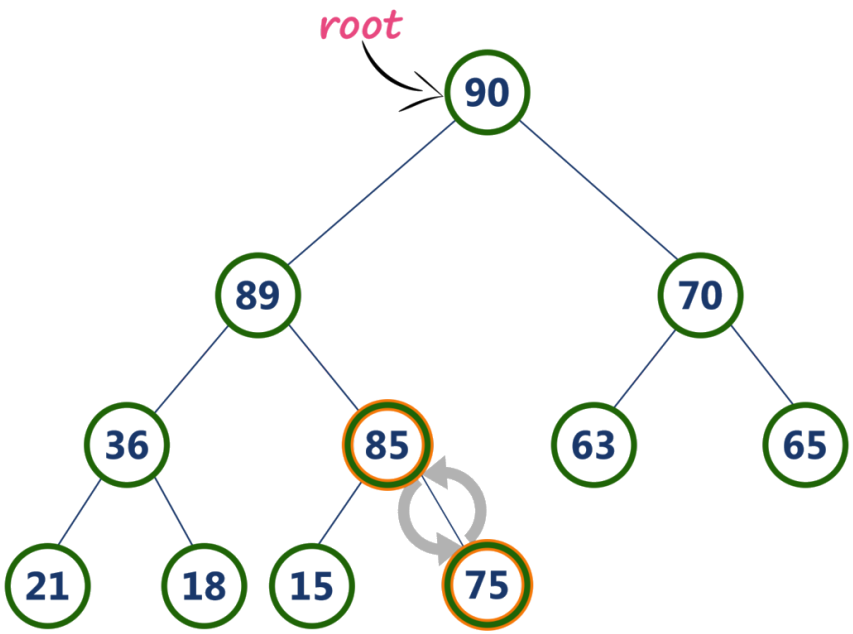
- **Step 1:** Insert the **newNode** with value 85 as **last leaf** from left to right. That means newNode is added as a right child of node with value 75. After adding max heap is as follows...



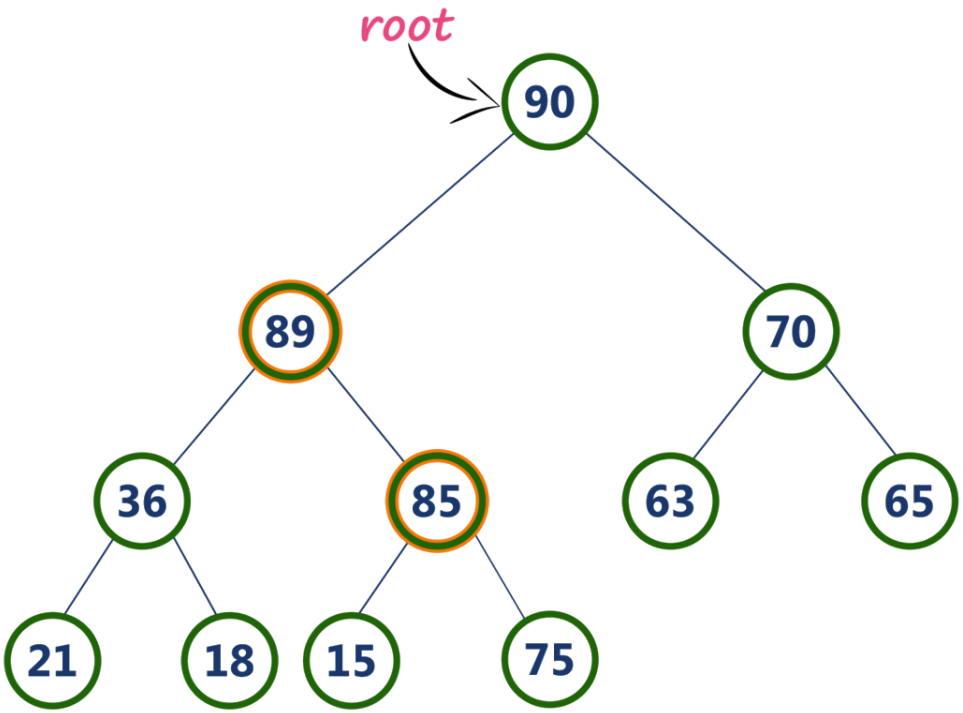
- **Step 2:** Compare **newNode value (85)** with its **Parent node value (75)**. That means **85 > 75**



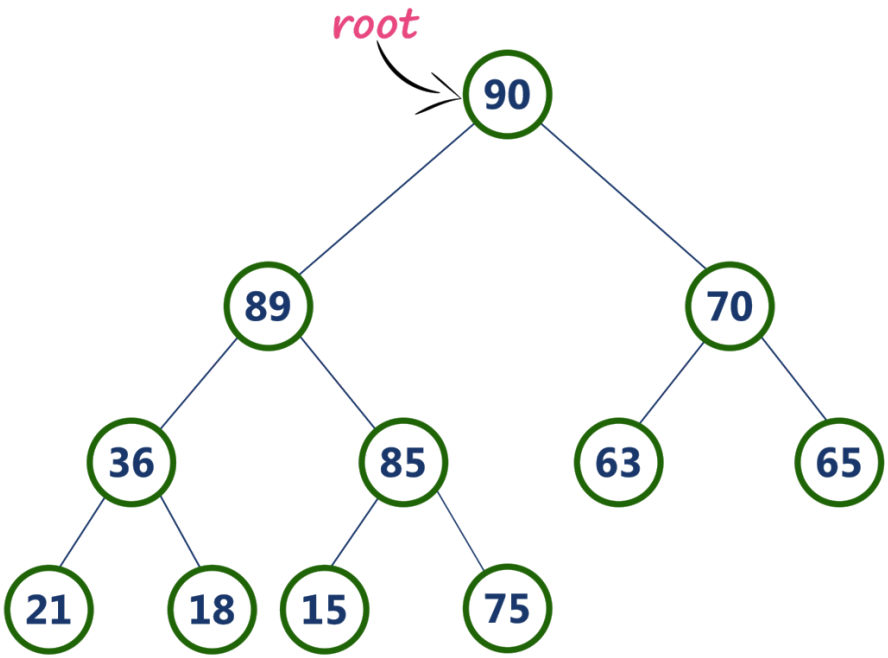
- **Step 3:** Here **newNode value (85) is greater** than its **parent value (75)**, then **swap** both of them. After wsapping, max heap is as follows...



Step 4: Now, again compare newNode value (85) with its parent node value (89).



- Here, newNode value (85) is smaller than its parent node value (89). So, we stop insertion process. Finally, max heap after insertion of a new node with value 85 is as follows...



- Animation video for insertion operation in Max Heap

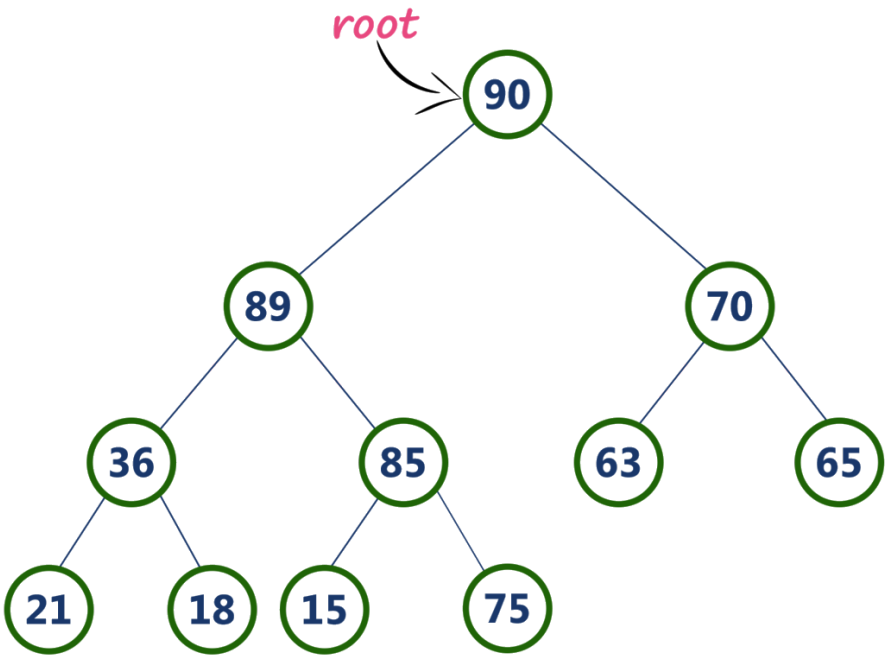
- **Note** − In Min Heap construction algorithm, we expect the value of the parent node to be less than that of the child node.

Deletion Operation in Max Heap

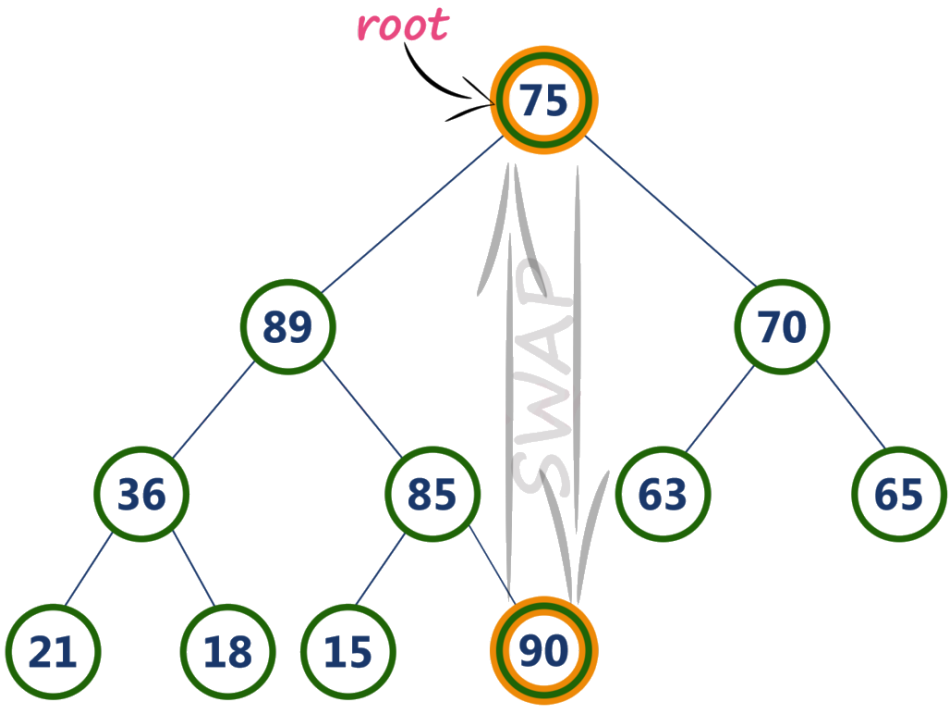
- Deletion in Max (or Min) Heap always happens at the root to remove the Maximum (or minimum) value.

- Deleting root node from a max heap is title difficult as it disturbing the max heap properties. We use the following steps to delete root node from a max heap...
  - **Step 1: Swap** the **root** node with **last** node in max heap
  - **Step 2: Delete** last node.
  - **Step 3:** Now, compare **root value** with its **left child value**.
  - **Step 4:** If **root value is smaller** than its left child, then compare **left child** with its **right sibling**. Else goto **Step 6**
  - **Step 5:** If **left child value is larger** than its **right sibling**, then **swap root** with **left child**. otherwise **swap root** with its **right child**.
  - **Step 6:** If **root value is larger** than its left child, then compare **root value** with its **right child** value.
  - **Step 7:** If **root value is smaller** than its **right child**, then **swap root** with **rith child**. otherwise **stop the process**.
  - **Step 8:** Repeat the same until root node is fixed at its exact position.

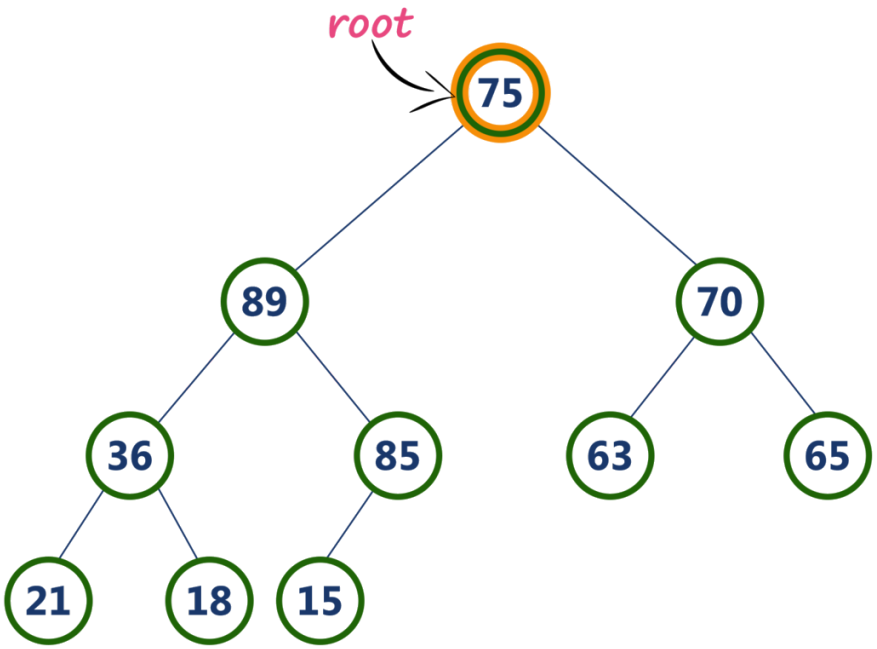
- Consider the following max heap. **Delete root node (90) from the max heap.**



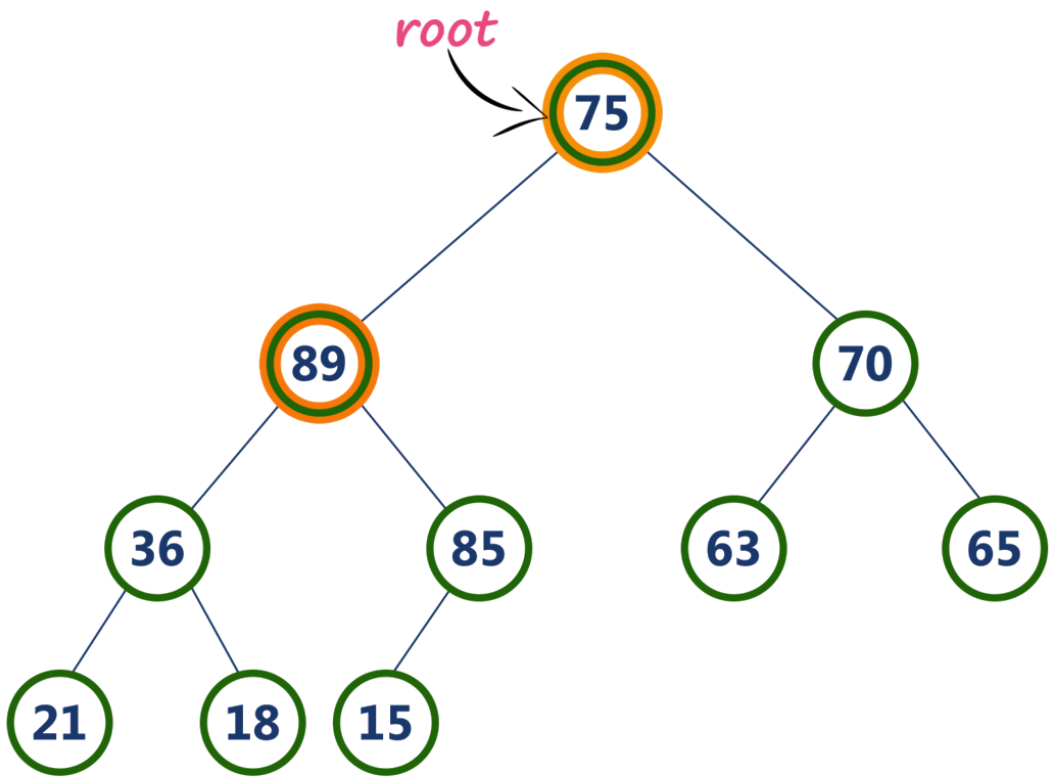
- **Step 1: Swap** the **root node (90)** with **last node 75** in max heap After swapping max heap is as follows...



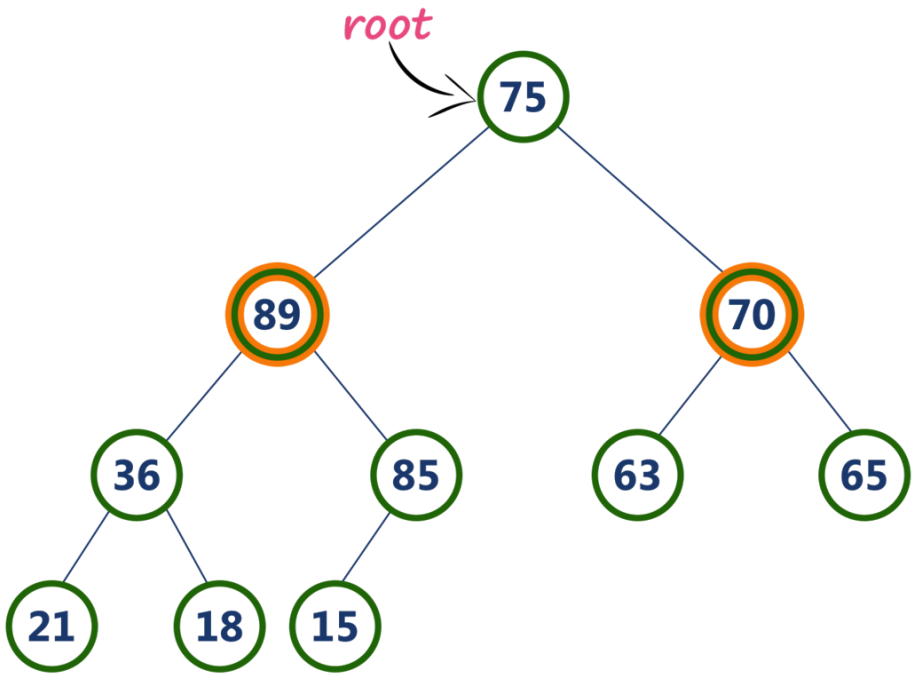
- **Step 2: Delete** last node. Here node with value 90. After deleting node with value 90 from heap, max heap is as follows...



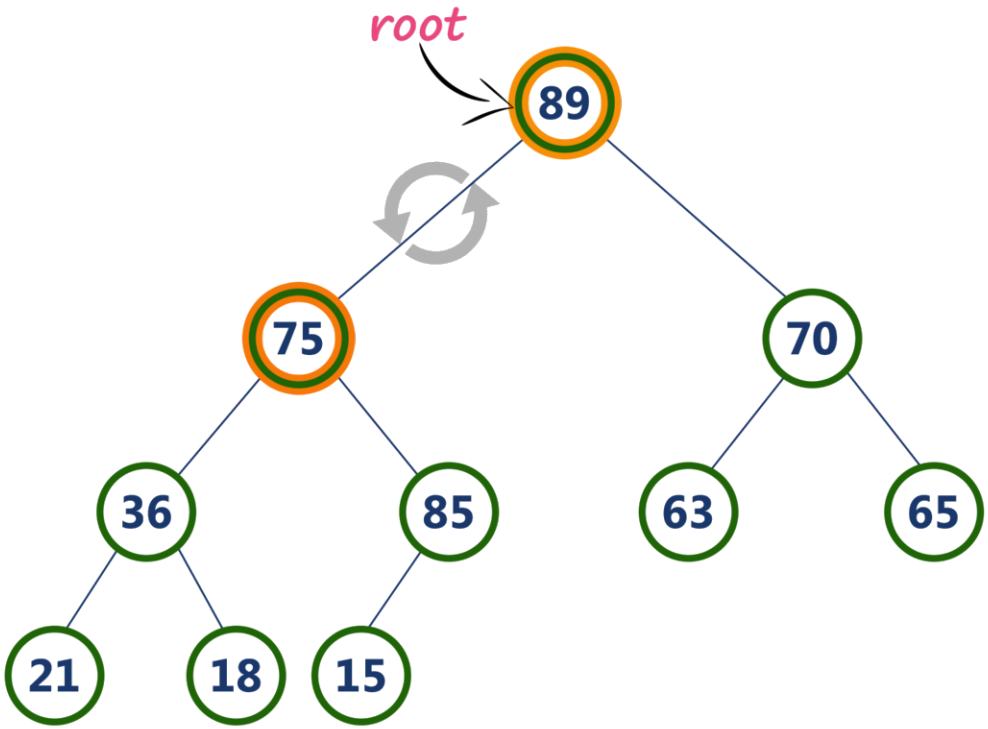
➢ **Step 3:** Compare **root node (75)** with its **left child (89)**.



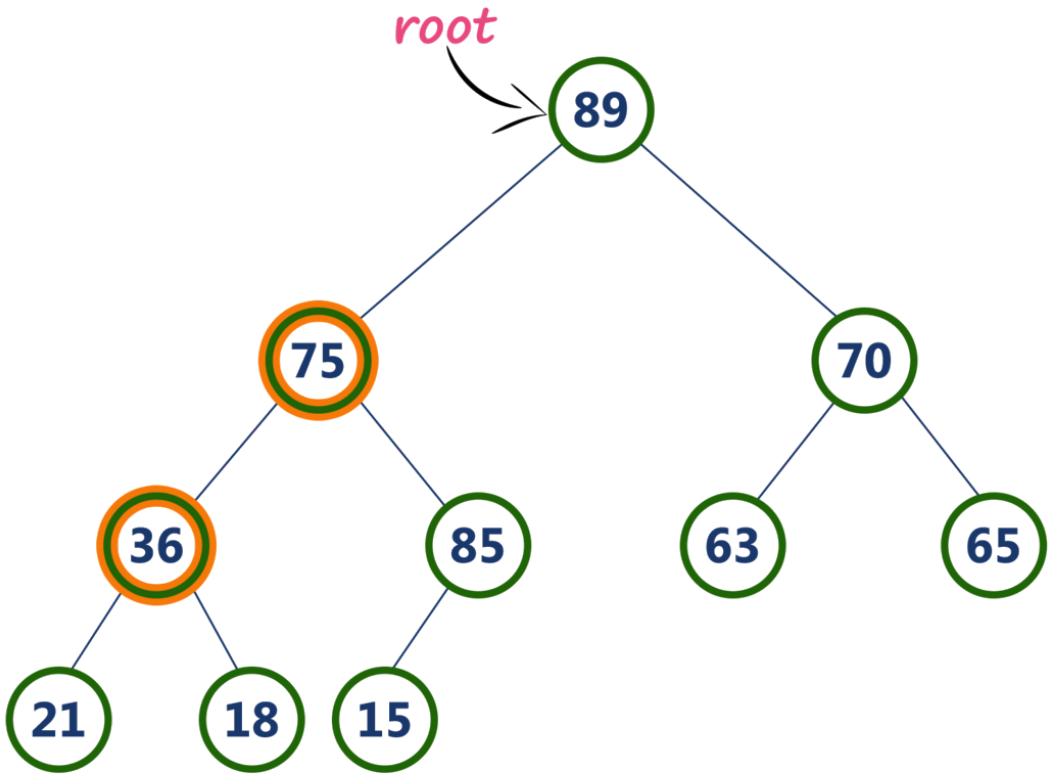
- Here, **root value (75) is smaller** than its left child value (89). So, compare left child (89) with its right sibling (70).



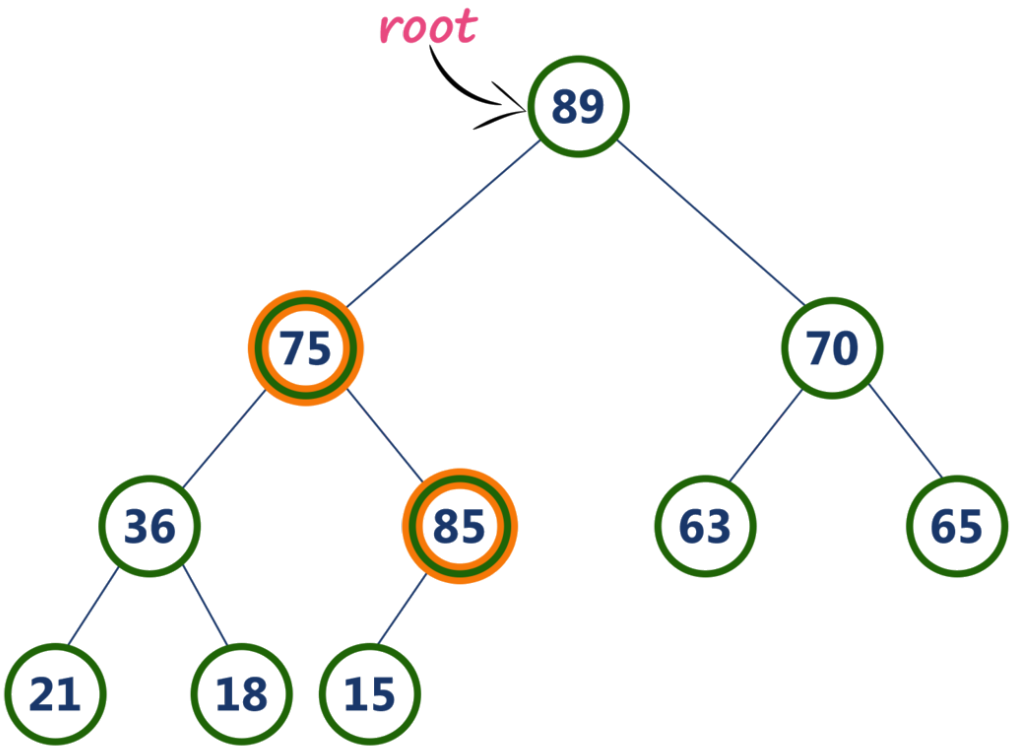
- **Step 4:** Here, **left child value (89) is larger** than its **right sibling (70)**, So, **swap root (75)** with **left child (89)**.



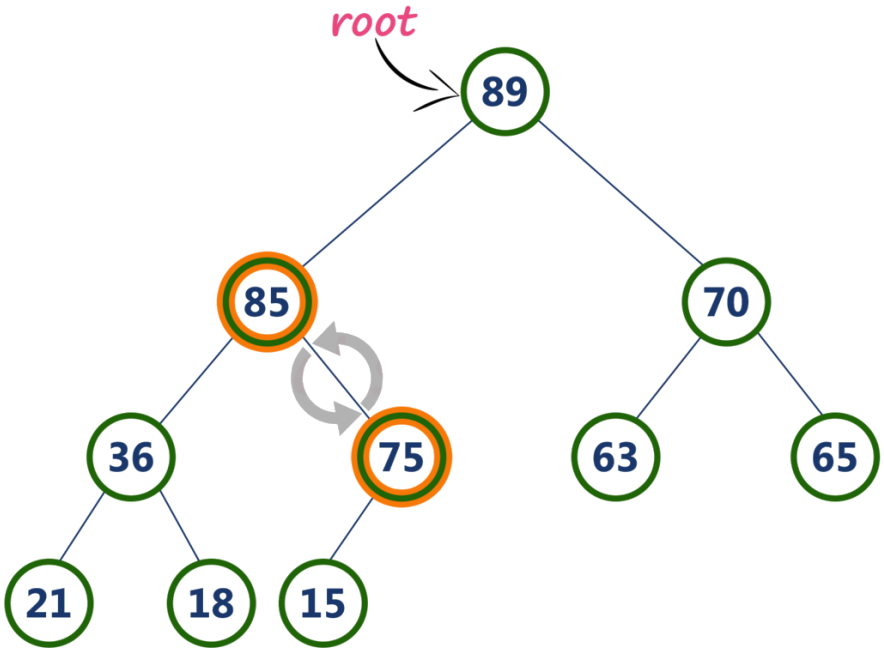
➢ **Step 5:** Now, again compare **75** with its **left child (36)**.



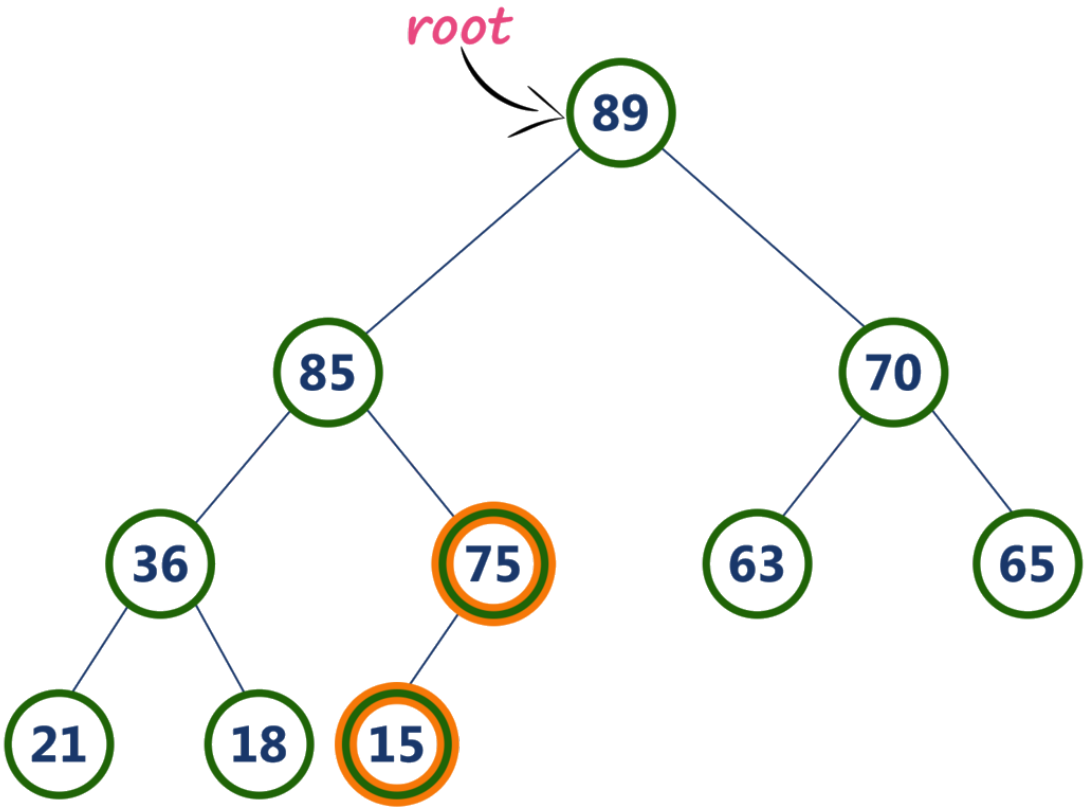
- Here, node with value **75** is larger than its left child. So, we compare node with value **75** is compared with its right child **85**.



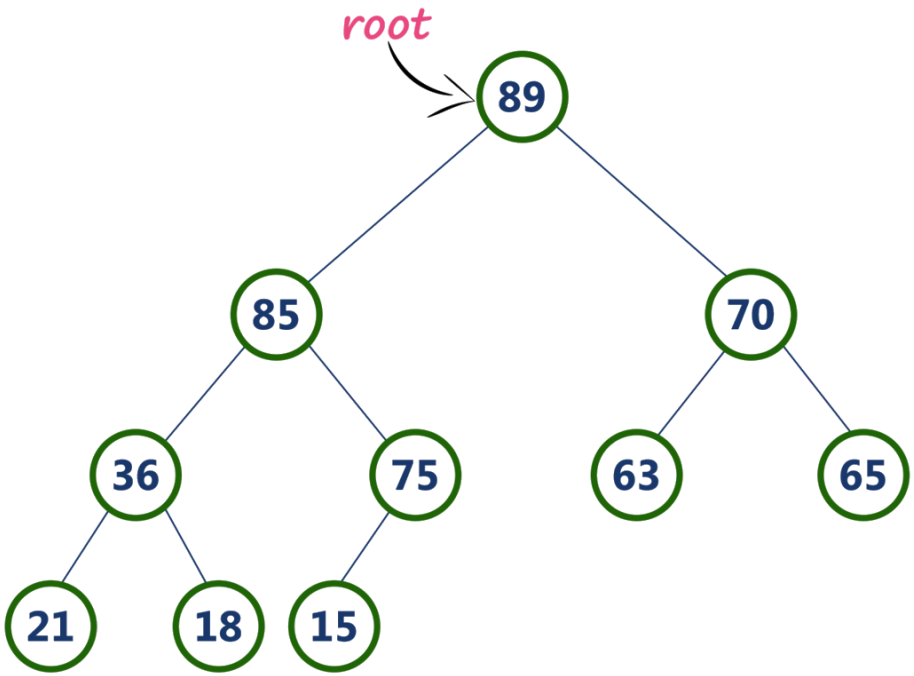
- **Step 6:** Here, node with value **75** is smaller than its **right child (85)**. So, we swap both of them. After swapping max heap is as follows...



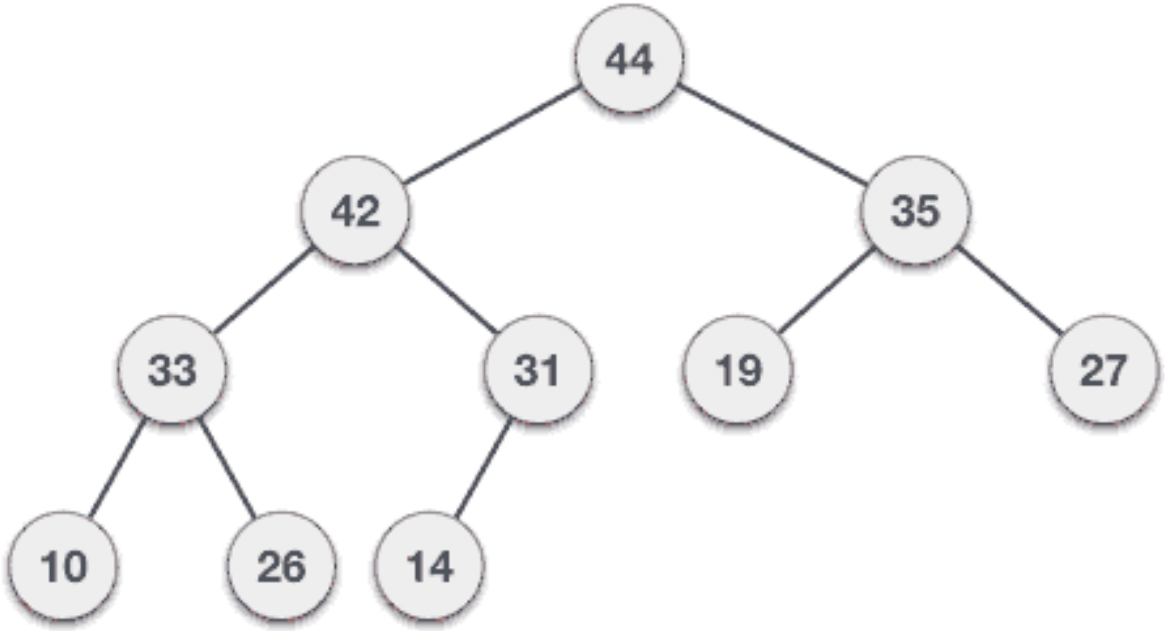
- **Step 7:** Now, compare node with value **75** with its left child (**15**).



- Here, node with value **75** is larger than its left child (**15**) and it does not have right child. So we stop the process.
- Finally, max heap after deleting root node (**90**) is as follows...



- Animation video for deletion operation in Max Heap



- **Note** − In Min Heap deletion algorithm, we expect the value of the parent node to be less than that of the child node.